

ESTADÍSTICAS DE UNA ENCUESTA UNIVERSITARIA

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Pregunte inicialmente cuántos estudiantes participarán. Por cada estudiante solicitar:

- Edad.
- Semestre.
- Horas de estudio por día.

Validaciones:

- Edad: 16 a 80.
- Semestre: 1 a 10.
- Horas de estudio: 0 a 24.

Determinar:

- Edad promedio.
- Horas promedio de estudio.
- Estudiante con mayor cantidad de horas.
- Estudiantes que estudian menos de 2 horas.
- Cantidad de estudiantes por semestre.

Para el último requisito deberán utilizar ciclos anidados o una estrategia equivalente explicada por el estudiante.

Criterios de implementación

Cada estudiante se identifica por su orden de registro: 1, 2, 3, etc. En un empate de horas se conserva al primero registrado. Se admite una cantidad positiva de participantes y se aceptan horas decimales con punto. Para “estudiantes que estudian menos de 2 horas” se informa la cantidad de estudiantes que cumplen la condición.

Se emplea un arreglo de contadores por semestre como estrategia equivalente a los ciclos anidados. No se agrega un menú porque el ejercicio consiste en realizar una encuesta y presentar sus resultados.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- n: cantidad de participantes; entero mayor que cero. La implementación usa enteros de 32 bits: de 1 a 2147483647. Ese máximo es un límite técnico del tipo de dato.
- edad: número entero entre 16 y 80, incluidos ambos límites.
- semestre: número entero entre 1 y 10, incluidos ambos límites.
- horas: número real entre 0 y 24, incluidos ambos límites; puede contener decimales.
- Se escribe un número por línea, sin espacios ni separadores de miles; se usa punto decimal. Texto vacío, letras, signos negativos, formatos incorrectos y valores fuera del rango se rechazan.

1.2 Proceso

- Solicitar n mediante LeerDato y repetir su lectura hasta que sea un entero válido y positivo.
- Inicializar sumaEdades, sumaHoras, menosDos y estudianteMayor en 0; mayorHoras en -1. Recorrer los semestres 1 a 10 para poner sus contadores en 0.
- Recorrer los n estudiantes. Mostrar su número y solicitar edad, semestre y horas; validar cada dato antes de continuar.
- Acumular edad en sumaEdades y horas en sumaHoras. Aumentar cantidad[semestre] en 1.
- Si horas > mayorHoras, actualizar mayorHoras y guardar el número del estudiante. El signo > conserva al primero cuando hay empate.
- Si horas < 2, aumentar menosDos en 1. Exactamente 2 horas no cumple esta condición.
- Calcular promedioEdad = sumaEdades / n y promedioHoras = sumaHoras / n. n es positivo, por lo que no hay división para cero.
- Mostrar los resultados y recorrer los diez semestres para presentar sus cantidades, incluidos los ceros. Solo redondear los resultados al mostrarlos.

1.3 Salida de datos

- Solicitudes de datos, número del estudiante en registro y mensajes de error cuando la entrada no es válida.
- Edad promedio y horas promedio de estudio, presentadas con dos decimales en Java y C++.
- Número del estudiante con más horas y su cantidad de horas diarias.
- Cantidad de estudiantes que estudian menos de 2 horas al día.
- Cantidad de estudiantes de cada semestre, del 1 al 10.

2. ALGORITMO

1. Inicio.

2. Declarar las variables de entrada, los acumuladores, los contadores, los promedios y el arreglo cantidad de 11 posiciones; se usarán las posiciones 1 a 10.

3. Solicitar n. Si el formato no es numérico, no es entero o está fuera del rango permitido, mostrar un error y repetir la lectura.

4. Inicializar sumaEdades = 0, sumaHoras = 0, menosDos = 0, mayorHoras = -1 y estudianteMayor = 0.

5. Para cada semestre s desde 1 hasta 10, asignar cantidad[s] = 0.

6. Asignar i = 0. Si i >= n, continuar en el paso 14.

7. Mostrar el número del estudiante i + 1.

8. Solicitar edad y repetir su lectura hasta obtener un entero de 16 a 80.

9. Solicitar semestre y repetir su lectura hasta obtener un entero de 1 a 10.

10. Solicitar horas y repetir su lectura hasta obtener un número de 0 a 24.

11. Sumar edad a sumaEdades y horas a sumaHoras; aumentar cantidad[semestre] en 1.

12. Si horas > mayorHoras, asignar mayorHoras = horas y estudianteMayor = i + 1. Si horas < 2, aumentar menosDos en 1.

13. Aumentar i en 1 y regresar a la comprobación del paso 6, sin reinicializar i.

14. Calcular promedioEdad = sumaEdades / n y promedioHoras = sumaHoras / n.

15. Mostrar ambos promedios, el estudiante con más horas, sus horas y la cantidad que estudia menos de 2 horas.

16. Recorrer s desde 1 hasta 10 y mostrar s y cantidad[s].

17. Fin.

3. PSEUDOCÓDIGO

Subproceso de lectura y validación

La lectura se hace como texto para detectar letras o formatos incorrectos. Se admite una parte entera y, opcionalmente, un punto seguido de dígitos. Después se verifican el rango y la condición de número entero.

SubProceso valor <- LeerDato(mensaje, minimo, maximo, entero)

Definir mensaje, texto, caracter Como Cadena;

Definir minimo, maximo, valor, divisor Como Real;

Definir entero, valido, decimal, digito Como Logico;

Definir k Como Entero;

Repetir

 Escribir Sin Saltar mensaje;

 Leer texto;

 valor <- 0;

 divisor <- 1;

 decimal <- Falso;

 valido <- Longitud(texto) > 0;

 k <- 0;

 Mientras k < Longitud(texto) Y valido Hacer

 caracter <- Subcadena(texto, k, k);

 digito <- caracter = "0" O caracter = "1";

 digito <- digito O caracter = "2" O caracter = "3";

 digito <- digito O caracter = "4" O caracter = "5";

 digito <- digito O caracter = "6" O caracter = "7";

 digito <- digito O caracter = "8" O caracter = "9";

 Si digito Entonces

 valor <- valor * 10 + ConvertirANumero(caracter);

 Si decimal Entonces

 divisor <- divisor * 10;

 FinSi

 SiNo

 Si caracter = "." Y No decimal Entonces

 Si k = 0 O k = Longitud(texto) - 1 Entonces

 valido <- Falso;

 FinSi

Subproceso de lectura y validación — continuación

```
    decimal <- Verdadero;
    SiNo
        valido <- Falso;
    FinSi
    FinSi
    k <- k + 1;
    FinMientras
    valor <- valor / divisor;
    valido <- valido Y valor >= minimo Y valor <= maximo;
    Si entero Y valor <> Trunc(valor) Entonces
        valido <- Falso;
    FinSi
    Si No valido Entonces
        Escribir "Dato invalido. Intente nuevamente.";
    FinSi
    Hasta Que valido
FinSubProceso
```

Ejemplo de conversión: para “1.5”, se forma valor = 15 y divisor = 10; al finalizar, $15 / 10 = 1,5$. Si se solicitan enteros, el dato se acepta solamente cuando valor = Trunc(valor).

Algoritmo principal

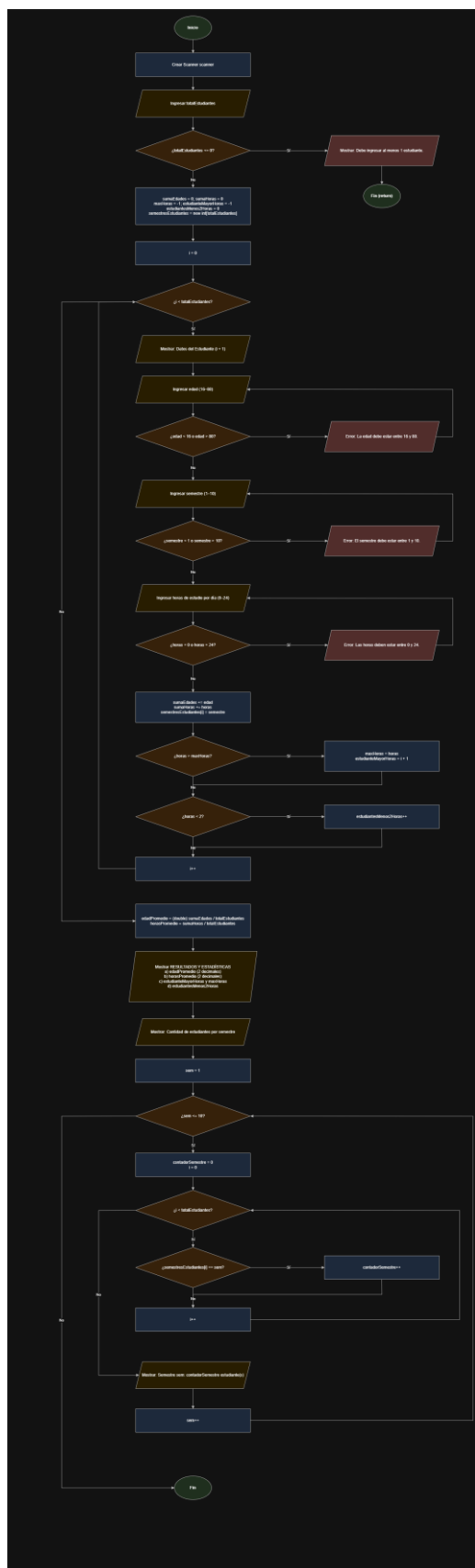
Algoritmo EncuestaUniversitaria

```
Definir n, edad, semestre, i, s Como Entero;
Definir estudianteMayor, menosDos Como Entero;
Definir cantidad Como Entero;
Dimension cantidad[11];
Definir horas, sumaEdades, sumaHoras Como Real;
Definir mayorHoras, promedioEdad, promedioHoras Como Real;
n <- LeerDato("Cantidad de estudiantes: ", 1, 2147483647, Verdadero);
sumaEdades <- 0;
sumaHoras <- 0;
mayorHoras <- -1;
estudianteMayor <- 0;
menosDos <- 0;
Para s <- 1 Hasta 10 Hacer
    cantidad[s] <- 0;
FinPara
```

Algoritmo principal — continuación

```
Para i <- 0 Hasta n - 1 Hacer
  Escribir "Estudiante ", i + 1;
  edad <- LeerDato("Edad: ", 16, 80, Verdadero);
  semestre <- LeerDato("Semestre: ", 1, 10, Verdadero);
  horas <- LeerDato("Horas por día: ", 0, 24, Falso);
  sumaEdades <- sumaEdades + edad;
  sumaHoras <- sumaHoras + horas;
  cantidad[semestre] <- cantidad[semestre] + 1;
  Si horas > mayorHoras Entonces
    mayorHoras <- horas;
    estudianteMayor <- i + 1;
  FinSi
  Si horas < 2 Entonces
    menosDos <- menosDos + 1;
  FinSi
FinPara
promedioEdad <- sumaEdades / n;
promedioHoras <- sumaHoras / n;
Escribir "Edad promedio: ", Redon(promedioEdad * 100) / 100;
Escribir "Horas promedio: ", Redon(promedioHoras * 100) / 100;
Escribir "Estudiante con mas horas: ", estudianteMayor;
Escribir "Mayor cantidad de horas: ", Redon(mayorHoras * 100) / 100;
Escribir "Estudiantes con menos de 2 horas: ", menosDos;
Para s <- 1 Hasta 10 Hacer
  Escribir "Semestre ", s, ": ", cantidad[s];
FinPara
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1. Datos variados y repetición de semestre

Tres estudiantes permiten verificar los promedios, el máximo, la condición de menos de 2 horas y dos registros en un mismo semestre.

Variable	Valor
n	3
edad, semestre, horas de E1	18; 1; 1,5
edad, semestre, horas de E2	20; 3; 4
edad, semestre, horas de E3	22; 3; 2
sumaEdades / sumaHoras finales	60 / 7,5
promedioEdad / promedioHoras	20,00 / 2,50
estudianteMayor / mayorHoras	2 / 4,00
menosDos	1
cantidad[1..10]	[1, 0, 2, 0, 0, 0, 0, 0, 0, 0]

Registro	sumaEdades	sumaHoras	Mayor (est.)	menosDos
Inicial	0	0	-1 (0)	0
E1	18	1,5	1,5 (1)	1
E2	38	5,5	4 (2)	1
E3	60	7,5	4 (2)	1

Operaciones paso a paso

E1: $0 + 18 = 18$; $0 + 1,5 = 1,5$; $\text{cantidad}[1] = 0 + 1 = 1$. Como $1,5 > -1$, el máximo pasa a E1; $1,5 < 2$ aumenta menosDos de 0 a 1.

E2: $18 + 20 = 38$; $1,5 + 4 = 5,5$; $\text{cantidad}[3] = 0 + 1 = 1$. Como $4 > 1,5$, el máximo pasa a E2; $4 < 2$ es falso.

E3: $38 + 22 = 60$; $5,5 + 2 = 7,5$; $\text{cantidad}[3] = 1 + 1 = 2$. Como $2 > 4$ y $2 < 2$ son falsos, el máximo y menosDos se mantienen.

Edad promedio = $60 / 3 = 20$. Horas promedio = $7,5 / 3 = 2,5$. Comprobación: $1 + 2 = 3$ estudiantes en los semestres.

Caso 2. Límites inferiores y superiores

Se verifican las edades 16 y 80, los semestres 1 y 10 y las horas 0 y 24. Todos los límites son válidos.

Variable	Valor
n	2
edad, semestre, horas de E1	16; 1; 0
edad, semestre, horas de E2	80; 10; 24
sumaEdades / sumaHoras finales	96 / 24
promedioEdad / promedioHoras	48,00 / 12,00
estudianteMayor / mayorHoras	2 / 24,00
menosDos	1
cantidad[1..10]	[1, 0, 0, 0, 0, 0, 0, 0, 0, 1]

Registro	sumaEdades	sumaHoras	Mayor (est.)	menosDos
Inicial	0	0	-1 (0)	0
E1	16	0	0 (1)	1
E2	96	24	24 (2)	1

Operaciones paso a paso

E1: $0 + 16 = 16$; $0 + 0 = 0$; $\text{cantidad}[1] = 0 + 1 = 1$. Como $0 > -1$, E1 queda registrado como máximo inicial; $0 < 2$ aumenta menosDos a 1.

E2: $16 + 80 = 96$; $0 + 24 = 24$; $\text{cantidad}[10] = 0 + 1 = 1$. Como $24 > 0$, el máximo pasa a E2; $24 < 2$ es falso.

Edad promedio = $96 / 2 = 48$. Horas promedio = $24 / 2 = 12$. Comprobación: $1 + 1 = 2$ estudiantes en los semestres.

Caso 3. Entradas inválidas, empate y exactamente 2 horas

Se repiten las lecturas incorrectas. Los dos estudiantes válidos estudian 2 horas: se conserva al primero como máximo y ninguno pertenece al grupo de menos de 2 horas.

Variable	Valor
n intentado → válido	0 (inválido) → 2
edad E1 intentada → válida	15, 81 (inválidas) → 18
semestre E1 intentado → válido	0, 11 (inválidos) → 2
horas E1 intentadas → válidas	abc, -1, 25 (inválidas) → 2
edad, semestre, horas de E2	20; 2; 2
sumaEdades / sumaHoras finales	38 / 4
promedioEdad / promedioHoras	19,00 / 2,00
estudianteMayor / mayorHoras	1 / 2,00
menosDos / errores de entrada	0 / 8
cantidad[1..10]	[0, 2, 0, 0, 0, 0, 0, 0, 0, 0]

Operaciones paso a paso

Los ocho datos inválidos generan mensajes de error. No se suman, no aumentan los contadores y no consumen un registro de estudiante.

E1 válido: $0 + 18 = 18$; $0 + 2 = 2$; $\text{cantidad}[2] = 0 + 1 = 1$. Como $2 > -1$, el máximo pasa a E1. Como $2 < 2$ es falso, $\text{menosDos} = 0$.

E2: $18 + 20 = 38$; $2 + 2 = 4$; $\text{cantidad}[2] = 1 + 1 = 2$. La comparación $2 > 2$ es falsa: el máximo sigue siendo E1. También es falso $2 < 2$.

Edad promedio = $38 / 2 = 19$. Horas promedio = $4 / 2 = 2$. Comprobación: $\text{cantidad}[2] = 2$, igual a n.

Resumen de los casos

Caso	Edad prom.	Horas prom.	Máximo	Menos de 2 h
1	20,00	2,50	E2: 4 h	1
2	48,00	12,00	E2: 24 h	1
3	19,00	2,00	E1: 2 h	0

6. CÓDIGO

JAVA

Guardar como EncuestaUniversitaria.java. El método leerDato reutiliza la misma validación para las cuatro entradas. Se emplea Scanner y las variables del programa principal se declaran antes del procesamiento.

```
import java.util.Scanner;
import java.util.Locale;

public class EncuestaUniversitaria {
    // Lee un dato y lo repite si no cumple las condiciones.
    static double leerDato(Scanner sc, String mensaje,
        double minimo, double maximo, boolean entero) {
        String texto;
        double valor;
        boolean valido;
        do {
            System.out.print(mensaje);
            texto = sc.nextLine();
            valido = texto.matches("[0-9]+([.][0-9]+)?");
            valor = -1;
            if (valido) {
                valor = Double.parseDouble(texto);
                valido = valor >= minimo && valor <= maximo;
                if (entero && valor % 1 != 0) {
                    valido = false;
                }
            }
            if (!valido) {
                System.out.println("Dato invalido. Intente nuevamente.");
            }
        } while (!valido);
        return valor;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

JAVA — continuación

```
int n, edad, semestre, i, s;
int estudianteMayor, menosDos;
int[] cantidad = new int[11];
double horas, sumaEdades, sumaHoras, mayorHoras;
double promedioEdad, promedioHoras;

n = (int) leerDato(sc, "Cantidad de estudiantes: ",
    1, Integer.MAX_VALUE, true);
sumaEdades = 0;
sumaHoras = 0;
mayorHoras = -1;
estudianteMayor = 0;
menosDos = 0;
for (s = 1; s <= 10; s++) {
    cantidad[s] = 0;
}
// i empieza en cero; el estudiante se identifica con i + 1.
for (i = 0; i < n; i++) {
    System.out.println("Estudiante " + (i + 1));
    edad = (int) leerDato(sc, "Edad: ", 16, 80, true);
    semestre = (int) leerDato(sc, "Semestre: ", 1, 10, true);
    horas = leerDato(sc, "Horas por dia: ", 0, 24, false);
    sumaEdades += edad;
    sumaHoras += horas;
    cantidad[semestre]++;
    if (horas > mayorHoras) {
        mayorHoras = horas;
        estudianteMayor = i + 1;
    }
    if (horas < 2) {
        menosDos++;
    }
}
promedioEdad = sumaEdades / n;
promedioHoras = sumaHoras / n;
System.out.printf(Locale.US, "Edad promedio: %.2f%n", promedioEdad);
System.out.printf(Locale.US, "Horas promedio: %.2f%n", promedioHoras);
System.out.println("Estudiante con mas horas: " + estudianteMayor);
System.out.printf(Locale.US, "Mayor cantidad de horas: %.2f%n", mayorHoras);
System.out.println("Estudiantes con menos de 2 horas: " + menosDos);
for (s = 1; s <= 10; s++) {
    System.out.println("Semestre " + s + ": " + cantidad[s]);
}
sc.close();
}
}
```

C++

Guardar como EncuestaUniversitaria.cpp. Compilar con C++17. Se mantiene la misma entrada, validación, orden de procesamiento, criterio de empate y salida que en Java.

```
#include <iostream>
#include <iomanip>
#include <string>
#include <sstream>
#include <regex>
#include <cmath>
#include <limits>
#include <locale>
using namespace std;

// Lee un dato y lo repite si no cumple las condiciones.
double leerDato(string mensaje, double minimo,
                double maximo, bool entero) {
    string texto;
    double valor;
    bool valido;
    do {
        cout << mensaje;
        getline(cin, texto);
        valido = regex_match(texto, regex("[0-9]+([.][0-9]+)?"));
        valor = -1;
        if (valido) {
            istringstream lectura(texto);
            lectura.imbue(locale::classic());
            valido = bool(lectura >> valor);
            valido = valido && valor >= minimo && valor <= maximo;
            if (entero && fmod(valor, 1.0) != 0) {
                valido = false;
            }
        }
    } while (!valido);
    return valor;
}

int main() {
```

C++ — continuación

```
int n, edad, semestre, i, s;
int estudianteMayor, menosDos;
int cantidad[11];
double horas, sumaEdades, sumaHoras, mayorHoras;
double promedioEdad, promedioHoras;

n = int(leerDato("Cantidad de estudiantes: ",
    1, numeric_limits<int>::max(), true));
sumaEdades = 0;
sumaHoras = 0;
mayorHoras = -1;
estudianteMayor = 0;
menosDos = 0;
for (s = 1; s <= 10; s++) {
    cantidad[s] = 0;
}
// i empieza en cero; el estudiante se identifica con i + 1.
for (i = 0; i < n; i++) {
    cout << "Estudiante " << i + 1 << endl;
    edad = int(leerDato("Edad: ", 16, 80, true));
    semestre = int(leerDato("Semestre: ", 1, 10, true));
    horas = leerDato("Horas por dia: ", 0, 24, false);
    sumaEdades += edad;
    sumaHoras += horas;
    cantidad[semestre]++;
    if (horas > mayorHoras) {
        mayorHoras = horas;
        estudianteMayor = i + 1;
    }
    if (horas < 2) {
        menosDos++;
    }
}
promedioEdad = sumaEdades / n;
promedioHoras = sumaHoras / n;
cout.imbue(locale::classic());
cout << fixed << setprecision(2);
cout << "Edad promedio: " << promedioEdad << endl;
cout << "Horas promedio: " << promedioHoras << endl;
cout << "Estudiante con mas horas: " << estudianteMayor << endl;
cout << "Mayor cantidad de horas: " << mayorHoras << endl;
cout << "Estudiantes con menos de 2 horas: " << menosDos << endl;
for (s = 1; s <= 10; s++) {
    cout << "Semestre " << s << ": " << cantidad[s] << endl;
}
return 0;
}
```

Verificación de los programas

La versión C++ se compiló con C++17 y se ejecutó con los tres casos de la prueba de escritorio. Los promedios, el estudiante con más horas, los contadores por semestre y los ocho rechazos del caso 3 coincidieron con los resultados calculados. El código Java se revisó, pero no se ejecutó porque no había un compilador Java disponible en el entorno de revisión.

Comandos para comprobar las versiones:

```
javac EncuestaUniversitaria.java  
java EncuestaUniversitaria
```

```
g++ -std=c++17 EncuestaUniversitaria.cpp -o encuesta  
./encuesta
```

Salida final comprobada del caso 1

```
Edad promedio: 20.00  
Horas promedio: 2.50  
Estudiante con mas horas: 2  
Mayor cantidad de horas: 4.00  
Estudiantes con menos de 2 horas: 1  
Semestre 1: 1  
Semestre 2: 0  
Semestre 3: 2  
Semestre 4: 0  
Semestre 5: 0  
Semestre 6: 0  
Semestre 7: 0  
Semestre 8: 0  
Semestre 9: 0  
Semestre 10: 0
```

La cantidad por semestre debe sumar n. Esta comprobación se cumple en los tres casos y permite detectar registros omitidos o contados más de una vez.